



System design

Email Client (e.g. Microsoft Outlook)

★ Premium



Medium



30 mins



235 done

Report an issue 🛠

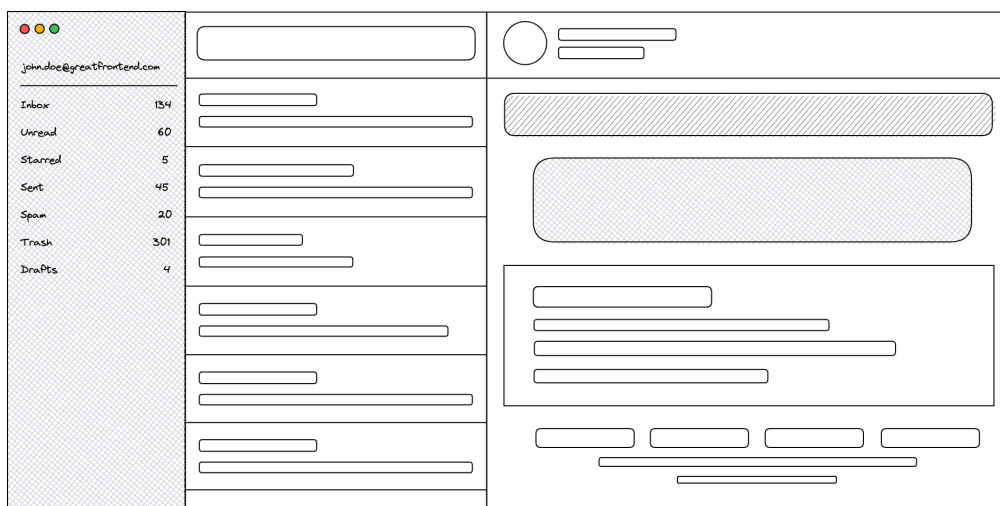
Asked at these companies



Microsoft

Question

Design a **desktop** email client that can send and receive email messages given an email provider service.



It's important to distinguish between **webmail** and email client apps. Websites like gmail.com, outlook.com, and Yahoo Mail, which allow you to use the browser to access email, are called webmail. Email clients are desktop apps that have to be installed on your computer and can usually be used even when offline. They usually allow access to multiple email services like Gmail, Outlook, iCloud, etc., and let you view messages from different services within the app.

Real-life examples

- Outlook for Windows/macOS
 - Apple Mail
 - Airmail
 - Mailspring
-

Requirements exploration

Before diving into the design, we scope the problem by clarifying the product surface: what the client must do, which platforms it targets, which accounts it connects to, and how it behaves offline.

What are the core functionalities needed?

- Sending email messages to an SMTP server.
- Retrieving email messages from an IMAP server.
- Accessing email messages already on the device.

What operating systems does the app need to support?

The popular ones: Windows, macOS, and Linux/Ubuntu.

What email services/accounts need to be supported?

We don't have to focus on that aspect for this question. Assume that the user can make authenticated requests to preconfigured SMTP/IMAP servers to send/retrieve emails successfully.

Many native desktop email clients like Apple Mail, Outlook, and Mailspring allow users to connect to multiple email services like iCloud Mail, Gmail, and Exchange to show emails from multiple services within the app. However, that is beyond the scope of this question.

Does the application need to work offline?

Yes, where possible. Outgoing email messages should be saved and sent out when the application comes back online. Users should still be allowed to browse and search for emails on the device even when they are offline.

Should emails between the same sender and topic be threaded?

Threading of message conversations would be good but is not required.

Exercise 1 of 2 Beta

A team is defining the first version of a desktop email client. Which requirements belong in it? Select all that apply.

- ☐ 1 Send mail through a configured provider, retrieve new mail, and read messages already stored on the device.
- ☐ 2 Require several providers to merge into one isolated multi-account inbox.
- ☐ 3 Require every message to be grouped into a conversation before it can be read.
- ☐ 4 Support the major desktop operating systems through a shared product design.

5

Let users browse and search synchronized mail offline, and preserve outgoing messages until connectivity returns.

☐

Reveal answer

Check answer ↩

Background knowledge

Email client applications are quite different from traditional web applications due to the fact that server requests are made using non-HTTP protocols like SMTP and IMAP. It is unlikely that interviewers will require candidates to be familiar with how the common email protocols work, so you can assume you are working with HTTP-based APIs for sending and retrieving emails.

Nevertheless, we'll cover some fundamental email system concepts for the sake of learning.

Parts of email systems

- **Mail User Agent (MUA):** An application where users can compose, send, receive, and read email messages. Other non-core functionalities include searching, flagging, address books, etc. These can be desktop applications with graphical user interfaces (Outlook, Apple Mail), or command line programs.
- **Mail Transfer Agent (MTA):** Software that transfers email messages from one host to another using the SMTP protocol. MTAs can exist on both users' devices and mail servers.
- **Mail Servers:** Computers that host the MTAs and store the email messages in a mailbox.
- **Mailbox:** A mailbox is a conceptual entity that does not necessarily pertain to storage and is identified by an email address. It contains email messages and typically exists on mail servers.

Mail transport protocols

If you have set up email clients before, you might have come across the terms SMTP, POP, and IMAP. SMTP is an outgoing email protocol used to send messages, while POP and IMAP are incoming email protocols that email servers support to allow clients to retrieve messages.

The benefit of having standardized protocols for sending and retrieving messages is that email services can send messages to each other and email clients can connect to any email service.

Simple Mail Transport Protocol (SMTP)

Simple Mail Transport Protocol is a protocol for sending email messages over the internet used by mail servers, MTAs, and MUAs (non-webmail).

SMTP uses a simple set of commands to transfer messages, including commands to authenticate the sender, specify the recipient(s), and send the message.

Nylas discussed SMTP relays in detail in their blog post "[SMTP vs. Web API: The Best Methods for Sending Email](#)". Here's an example SMTP conversation with an SMTP server over the command line. Lines starting with **S:** are sent from the server, and lines starting with **C:** are written by the user.

```
1  $ openssl s_client -connect smtp.example.com:465 -crlf
2
3  S: 220 smtp.example.com ESMTP Postfix
4  C: HELO relay.example.org
5  S: 250 Hello relay.example.org, I am glad to meet you
6  C: AUTH LOGIN
7  S: 334 VXNlcm5hbWU6
8  C: dXNlcm5hbWUuY29t # Username encoded in Base64
9  S: 334 UGFzc3dvcmQ6
10 C: bXlwYXNzd29yZA== # Password encoded in Base64
11 S: 235 Authentication succeeded
12 C: MAIL FROM:<bob@example.org>
13 S: 250 Ok
14 C: RCPT TO:<alice@example.com>
15 S: 250 Ok
16 C: RCPT TO:<theboss@example.com>
17 S: 250 Ok
18 C: DATA
19 S: 354 End data with <CR><LF>.<CR><LF>
```

```
20 C: From: "Bob Example" <bob@example.org>
21 C: To: "Alice Example" <alice@example.com>
22 C: Cc: theboss@example.com
23 C: Date: Tue, 15 Jan 2008 16:02:43 -0500
24 C: Subject: Test message
25 C:
26 C: Hello Alice.
27 C: This is a test message with 5 header fields and 4 lines in the message
28 C: Your friend,
29 C: Bob
30 C: .
31 S: 250 Ok: queued as 12345
32 C: QUIT
33 S: 221 Bye
34 {The server closes the connection}
```

The specification for SMTP can be found in [RFC5321](#).

Post Office Protocol (POP)

Post Office Protocol is a traditional standard protocol for accessing email messages on a mail server. Messages are kept on the server only until the first device accesses and downloads them. As the name suggests, once an email is downloaded, it is usually deleted from the server, just like how post offices act as temporary storage for physical mail before they get delivered to recipients.

POP3 is the most recent widely-used version of POP, but it is older than newer protocols like IMAP and has fewer features. POP is generally considered less desirable than IMAP as it is less flexible and does not allow for server-side searching or message flagging.

The specification for POP3 can be found in [RFC1939](#).

Internet Message Access Protocol (IMAP)

Internet Message Access Protocol is a standard protocol for accessing email messages on a mail server, with the latest version being IMAP4. IMAP allows users to retrieve and manage their email messages within webmail and email clients without having to download them to the local computer. IMAP also allows users to access their email from multiple devices and locations, and provides features such as server-side searching and message flagging.

IMAP addresses many of the shortcomings of POP, at the cost of server storage. Nylas published a [deep dive into IMAP](#) on their engineering blog which we highly recommend checking out.

The specification for IMAP4 can be found in [RFC3501](#).

POP vs IMAP

Here's a table comparing the POP (POP3) and IMAP (IMAP4) protocols.

Feature	POP3	IMAP
Source of truth	Client(s)	Server
No. of simultaneous clients	One	Multiple
No. of mailboxes	One	Multiple
Message downloading	Entire message	Individual parts
Message <u>flagging</u> (seen, answered, deleted)	No	Yes
Deleted from server after downloading	Yes by default	No
Server-side search	No	Yes
Server storage usage	Low	High

Reference: [IMAP vs. POP3: What's the Difference? Which One Should You Use?](#)

Today, users expect to access emails on multiple devices and view a consistent mailbox state across different client devices, so POP's model is outdated. The main advantage of POP is that less server storage space is needed, but that is usually not a concern these days since storage is relatively cheap.

IMAP is the prevalent email protocol in the current age, but many email clients still support retrieving emails from both IMAP and POP servers.

✔

Default to IMAP as the canonical retrieval protocol in your design
Modern users expect a consistent mailbox across phone, desktop, and webmail, which requires server-side state, flags, and multi-client access that POP3 does not offer. Treat POP3 support as an optional fallback for legacy accounts, not as the shape that drives the client architecture.

Email server configurations

Popular email services have the following configurations.

Service	SMTP	IMAP	POP
<u>Gmail</u>	smtp.gmail.com	imap.gmail.com	pop.gmail.com
<u>Outlook</u>	smtp-mail.outlook.com	outlook.office365.com (port 993)	outlook.office365.com (port 995)
<u>iCloud</u>	smtp.mail.me.com	imap.mail.me.com	Unsupported

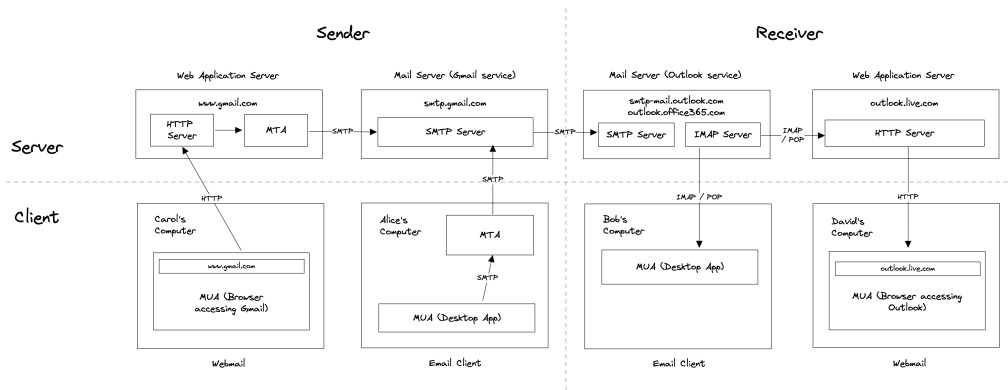
Email flow

Let's assume the following users with the respective roles, services, and types of clients:

User	Role	Service	Client type
Alice	Sender	Gmail	Desktop app
Bob	Receiver	Outlook	Desktop app
Carol	Sender	Gmail	Webmail

User	Role	Service	Client type
David	Receiver	Outlook	Webmail

We'll go through the flows detailing how messages are sent between the various types of clients.



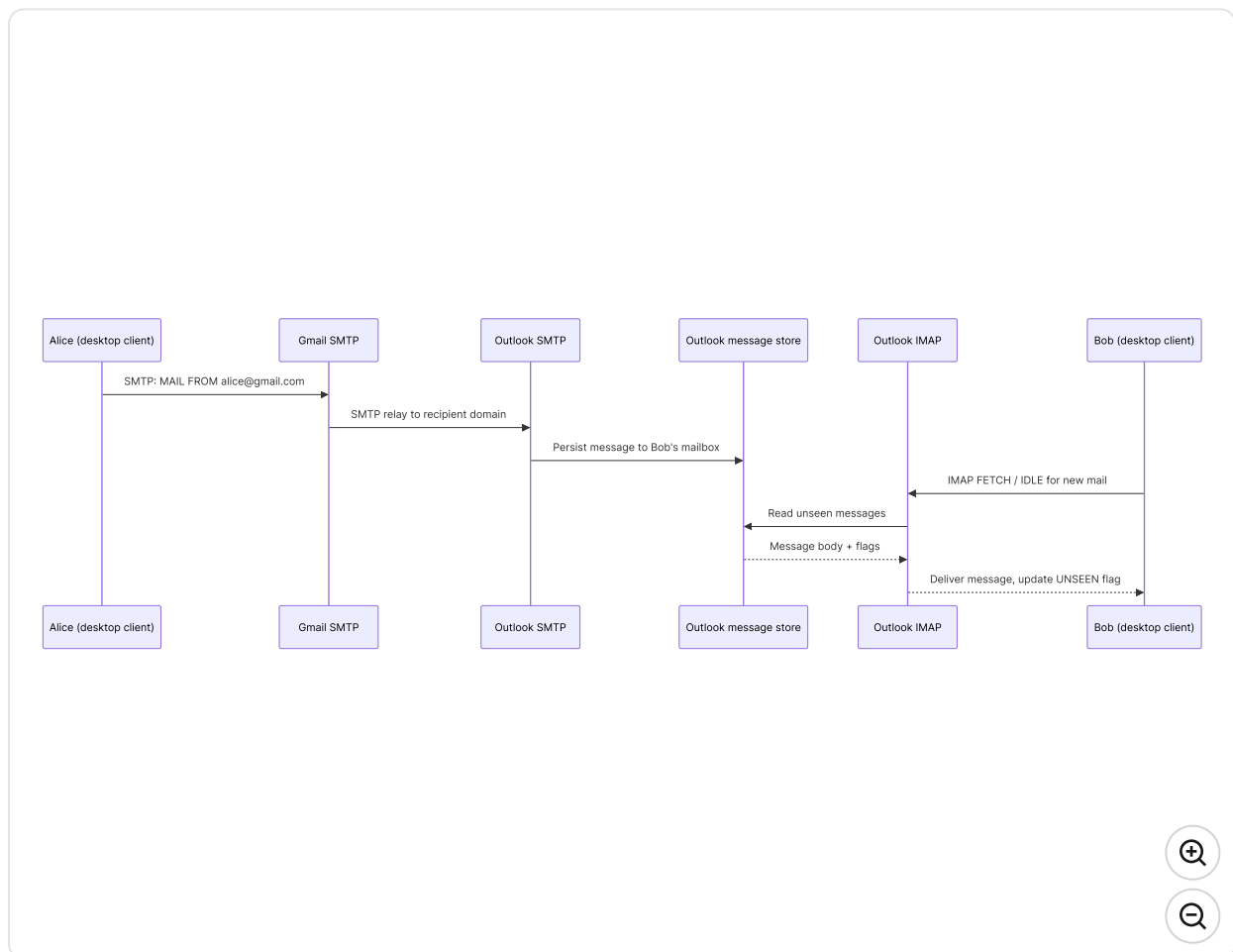
How email messages travel from senders to recipients

Important notes:

- Mail services can be running SMTP servers and IMAP/POP servers on the same machine or on different machines. The decision is not important for our discussion as long as the servers for a mail service have access to the same email message database.
- The arrow directions indicate the flow of the email message, not the origin of the request. IMAP/POP requests are initiated by clients, not pushed from the mail servers.
- Gmail's IMAP servers are not shown because in the above scenario, Gmail is only used for sending and not receiving.
- The DNS lookup stage for the MX (Mail Exchange) record is omitted for simplicity. SMTP servers use the domain name of the recipient's email address to look up DNS records for that domain (MX records in particular) to determine the address of the mail server.
- MTA is a fairly general term, but they all use SMTP to send messages. SMTP servers are MTAs.

Email client → Email client

1. Alice (alice@gmail.com) wants to send an email to Bob (bob@outlook.com).
2. Alice's email client desktop app sends the message to the MTA, software running on her computer.
3. Alice's computer's MTA sends the message to Gmail's SMTP server (`smtp.gmail.com`) over SMTP.
4. Gmail's SMTP servers send the message to Outlook's SMTP servers (`smtp-mail.outlook.com`) over SMTP, and the message is saved into Outlook's database.
5. Bob's email desktop client makes IMAP requests to Outlook's IMAP servers (`outlook.office365.com:993`).
6. Outlook's IMAP servers retrieve the message from the database and send it back to Bob's desktop client over IMAP.
7. Bob's email desktop client displays the new email message from Alice.



Email delivery from sender client to receiver client

Email client → Webmail

1. Alice (alice@gmail.com) wants to send an email to David (david@outlook.com).

2. Alice's email client desktop app sends the message to the MTA, software running on her computer.
3. Alice's computer's MTA sends the message to Gmail's SMTP server (`smtp.gmail.com`) over SMTP.
4. Gmail's SMTP servers send the message to Outlook's SMTP servers (`smtp-mail.outlook.com`) over SMTP, and the message is saved into Outlook's database.
5. David accesses Outlook webmail by visiting `https://outlook.live.com` in his browser.
6. The servers hosting `outlook.live.com` make IMAP requests to Outlook's IMAP servers (`outlook.office365.com:993`).
7. Outlook's IMAP servers retrieve the message from the database and send it back to `outlook.live.com`.
8. The servers hosting `outlook.live.com` send the response to David's browser over HTTP.
9. David's browser displays the new email message from Alice.

Webmail → Email client

1. Carol (`carol@gmail.com`) wants to send an email to Bob (`bob@outlook.com`).
2. Carol accesses Gmail webmail by visiting `https://www.gmail.com` in her browser.
3. Carol sends out an email message from Gmail webmail, which makes an HTTP request to `gmail.com` servers.
4. `gmail.com` servers use their MTA software to send the message to Gmail's SMTP server (`smtp.gmail.com`) over SMTP.
5. Gmail's SMTP servers send the message to Outlook's SMTP servers (`smtp-mail.outlook.com`) over SMTP, and the message is saved into Outlook's database.
6. Bob's email desktop client makes IMAP requests to Outlook's IMAP servers (`outlook.office365.com:993`).
7. Outlook's IMAP servers retrieve the message from the database and send it back to Bob's desktop client over IMAP.
8. Bob's email desktop client displays the new email message from Carol.

Webmail → Webmail

1. Carol (carol@gmail.com) wants to send an email to David (david@outlook.com).
2. Carol accesses Gmail webmail by visiting <https://www.gmail.com> in her browser.
3. Carol sends out an email message from Gmail webmail, which makes an HTTP request to gmail.com servers.
4. gmail.com servers use their MTA software to send the message to Gmail's SMTP server (smtp.gmail.com) over SMTP.
5. Gmail's SMTP servers send the message to Outlook's SMTP servers (smtp-mail.outlook.com) over SMTP, and the message is saved into Outlook's database.
6. David accesses Outlook webmail by visiting <https://outlook.live.com> in his browser.
7. The servers hosting outlook.live.com make IMAP requests to Outlook's IMAP servers (outlook.office365.com:993).
8. Outlook's IMAP servers retrieve the message from the database and send it back to outlook.live.com over IMAP.
9. The servers hosting outlook.live.com send the response to David's browser over HTTP.
10. David's browser displays the new email message from Carol.

Types of email systems

Email systems can be broadly categorized into the following types:

Store and forward servers

Store and forward email servers usually run on POP, where messages are kept on the server only until a user first accesses and downloads them. It is a simple and straightforward design.

Advantages

- Messages do not remain on the server for long (until a client accesses them), and the server doesn't need to do much processing on them.

- Client devices usually store the downloaded messages, so users can still access old messages even when there is no internet connection or when the mail server is unavailable due to outages.
- The server does not require much storage space as messages are deleted after downloading.

Disadvantages

- Since messages are stored locally, if the server is accessed from multiple client devices, there's no consistent view of all the messages.
- Users are responsible for backing up and restoring their messages. Without any backup, messages will be lost forever if devices are damaged or stolen.
- Functionality such as searching/sorting of messages has to be done locally on the device, which can be computationally intensive for mailboxes containing a large number of messages. In the case of messages being stored across multiple devices, searching is limited to the messages on the current device, which can be inconvenient.

Server-only mailboxes

In such a system, servers act as the source of truth for the messages, and even after clients download the messages, the server retains them. Clients do not cache or persist the messages after downloading them. Such servers can run on IMAP, and webmail is a common example of this model.

Advantages

- All devices have a consistent view of the mailboxes.
- Backups can be done by the email service providers without users going through any hassle.
- Functionality that can be computationally intensive or difficult for clients to perform can be done on the server. Less powerful devices like mobile phones will benefit from this.

Disadvantages

- Requires an internet connection to view the messages.

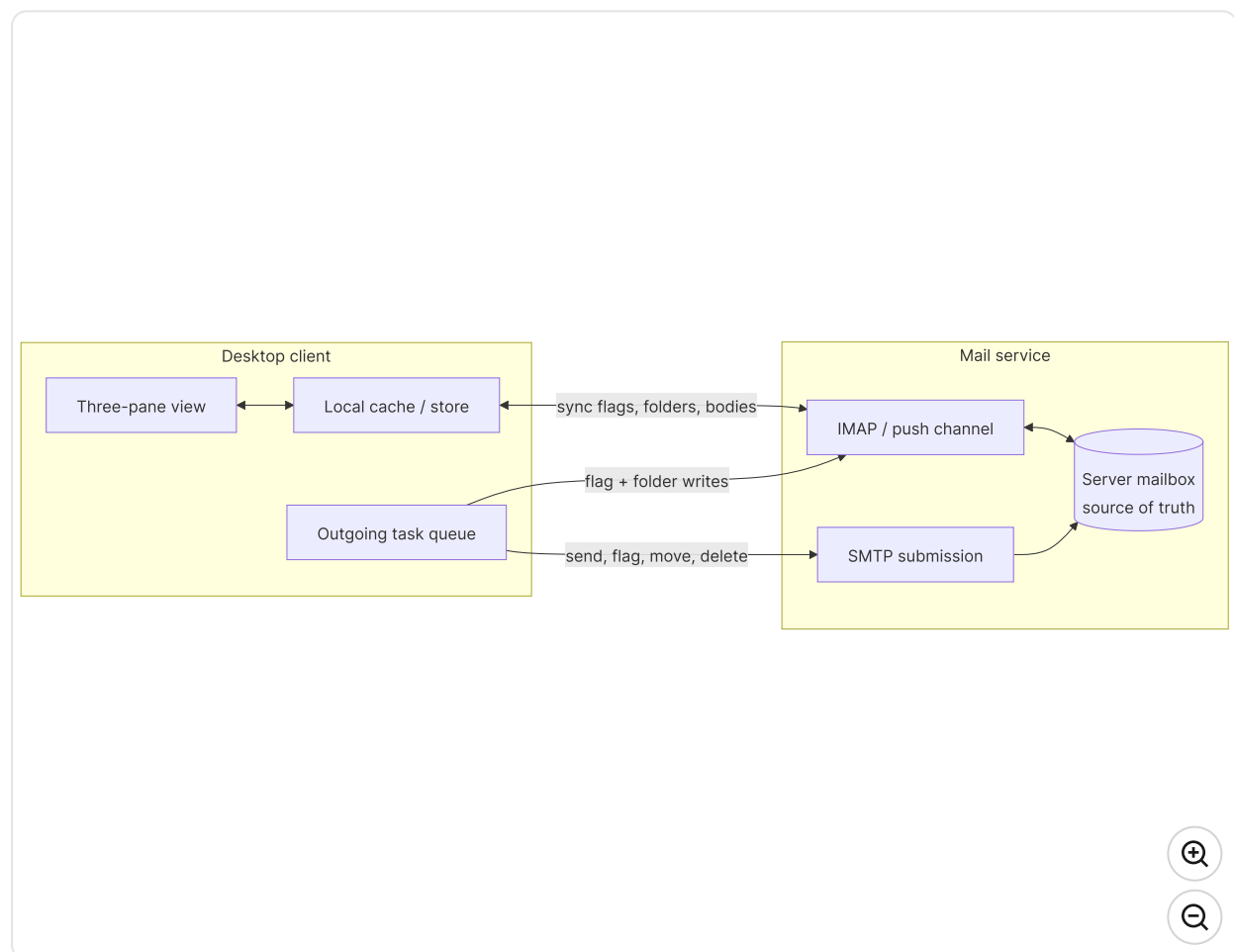
- Sufficient server storage space is required as it is the source of truth for messages.

Server mailbox with client-side cache

A hybrid model combines the best characteristics of both the store and forward servers and the server-only mailbox by having a permanent server mailbox while clients cache/persist the downloaded messages. Most desktop email clients operate using this model, which is the most complex of the three.

✓ Desktop clients should use the server-with-client-cache hybrid by default

This is the model Outlook, Apple Mail, and Mailspring actually ship. The server stays the source of truth for cross-device consistency and backups, while the local cache unlocks offline browsing, search, and snappy UI at the cost of real synchronization logic the client must own end-to-end.



Server mailbox with client-side cache hybrid model

Advantages

- Most advantages of the store and forward model:

- Can access messages even when offline or when the email server is unavailable.
- Advantages of server-only mailboxes:
 - All devices have a consistent view of the mailboxes.
 - Backup done by email service providers.
 - Leverage server-side features like searching.

Disadvantages

- Complex synchronization logic between client and server.
- Sufficient server storage space is required, as the server is still the source of truth for messages.

Reference: *NinjaMail: the Design of a High-Performance Clustered, Distributed E-mail System*

Exercise 1 of 3 Beta

Users expect read flags, folders, and searches to stay consistent across desktop, phone, and webmail. Which retrieval model should drive the client architecture?

- 1 Use POP as the canonical model because downloading and deleting messages from the server minimizes storage.
- 2 Use SMTP for both sending and retrieving so every client participates in the same transport protocol.
- 3 Use IMAP with the server mailbox as the source of truth and treat POP support as an optional legacy path.



Reveal answer

Check answer ←

Native HTML apps

- How to build native apps
- Talk about examples (Electron/Nativefier/Tauri) and their differences:

Architecture / high-level design

The first architectural fork is where the client runs, either as a native desktop app or in the browser, since that choice cascades into rendering, storage, and how mutations reach the mail server.

Desktop client vs Webmail

- Separate into Client app vs Isomorphic core.
- Abstract out database layer so can select db depending on runtime environment.
- Values of a native desktop app:
 - Menubar
 - Notifications
 - Keyboard shortcuts that don't conflict with the browser's
 - Badge

Flux/Redux architecture

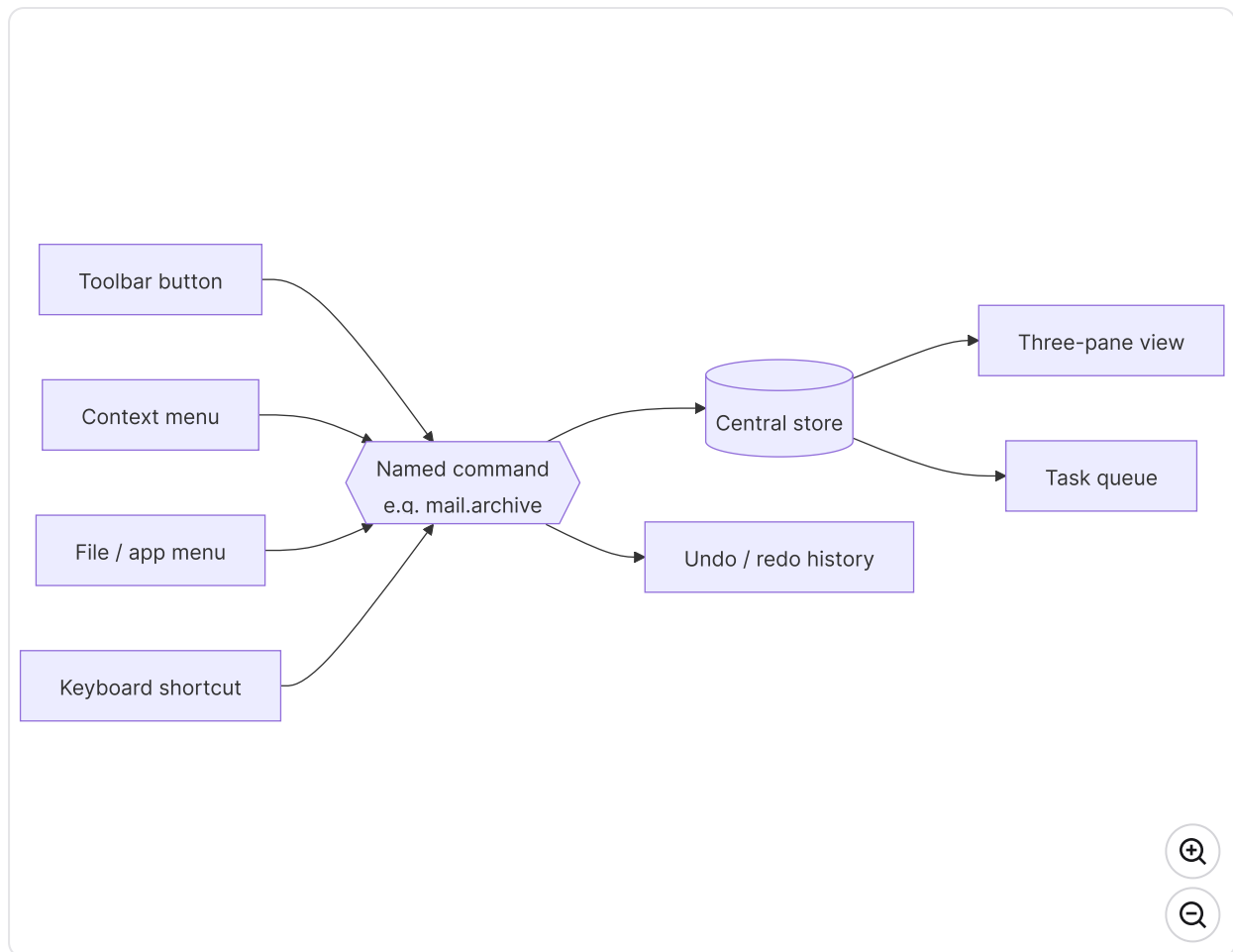
Use a Flux/reducer architecture or Command query request separation. There are many different actions in the application that can be abstracted out as application-wide commands and have multiple trigger sources (e.g. UI element interaction, File menu interaction, keyboard shortcut).

- Central store. Many parts of the UI rely on the same data store.
- Namespaced commands for better organization and lower chance of collision.
- Easily implement undo/redo.
- Mailspring's list of actions

★ Route every trigger source through the same named commands

The same action, such as archive, mark read, send, or delete, can be fired from a toolbar button, a context menu, a File-menu item, or a keyboard shortcut.

Dispatching through a single named command per action keeps behavior consistent across entry points, makes undo/redo a central concern instead of a per-surface hack, and gives you a natural boundary for logging, analytics, and extensions.



Unified command dispatch across trigger sources

Server-side rendering vs Client-side rendering

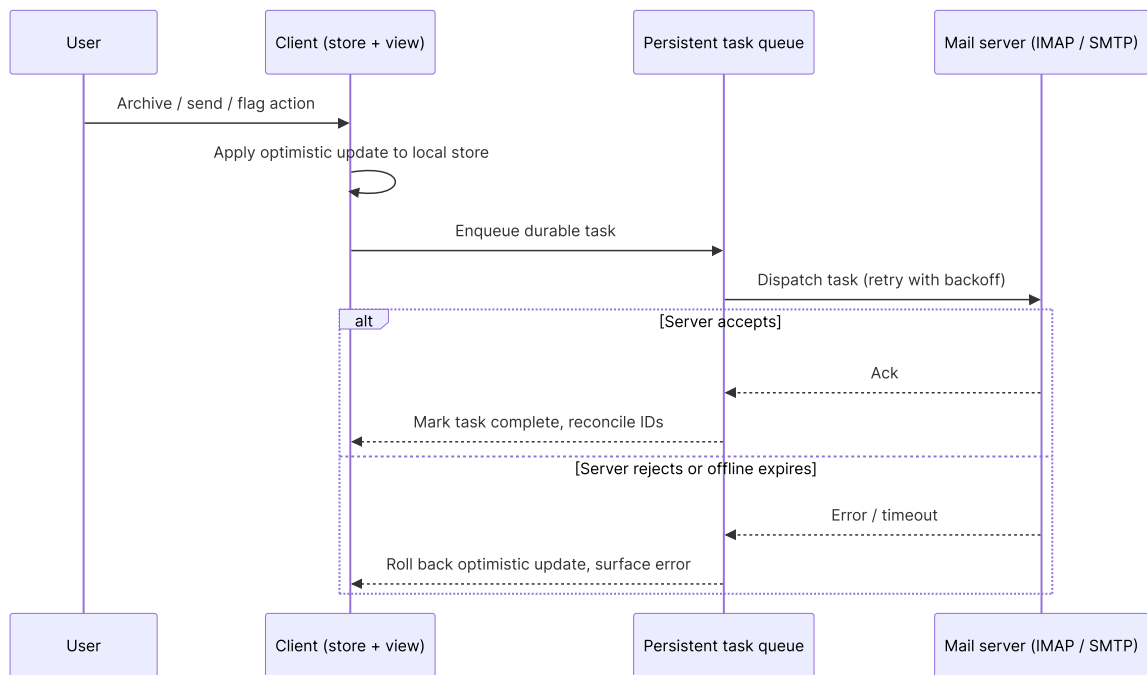
- CSR because it's an app.

Task queue

- Mutable operations will update the local data store immediately and queue changes to sync back to your mail provider.

★ Optimistically mutate the local store, then queue work for the server

Reading, archiving, deleting, and sending should feel instant: update the local cache first and enqueue a durable task for the mail server. The queue must survive restarts, retry with backoff when offline, and know how to reconcile or roll back when the server rejects a change; without that, "sent" messages silently vanish and flag state drifts between devices.



Optimistic mutation with durable task queue

Exercise 1 of 5 Beta

The same email product will ship as a desktop client and as webmail, but their storage and platform capabilities differ. Which boundary avoids duplicating core behavior?

- 1 Treat the rendered mailbox DOM as the cross-runtime core and attach platform-specific event handlers around it.
- 2 Share commands, state transitions, and synchronization logic in an isomorphic core while adapting the shell and database layer to each runtime.
- 3 Implement separate command, synchronization, and storage logic for desktop and webmail so each can optimize independently.
- 4 Use the browser's storage implementation unchanged in the desktop app so both clients share an identical persistence layer.

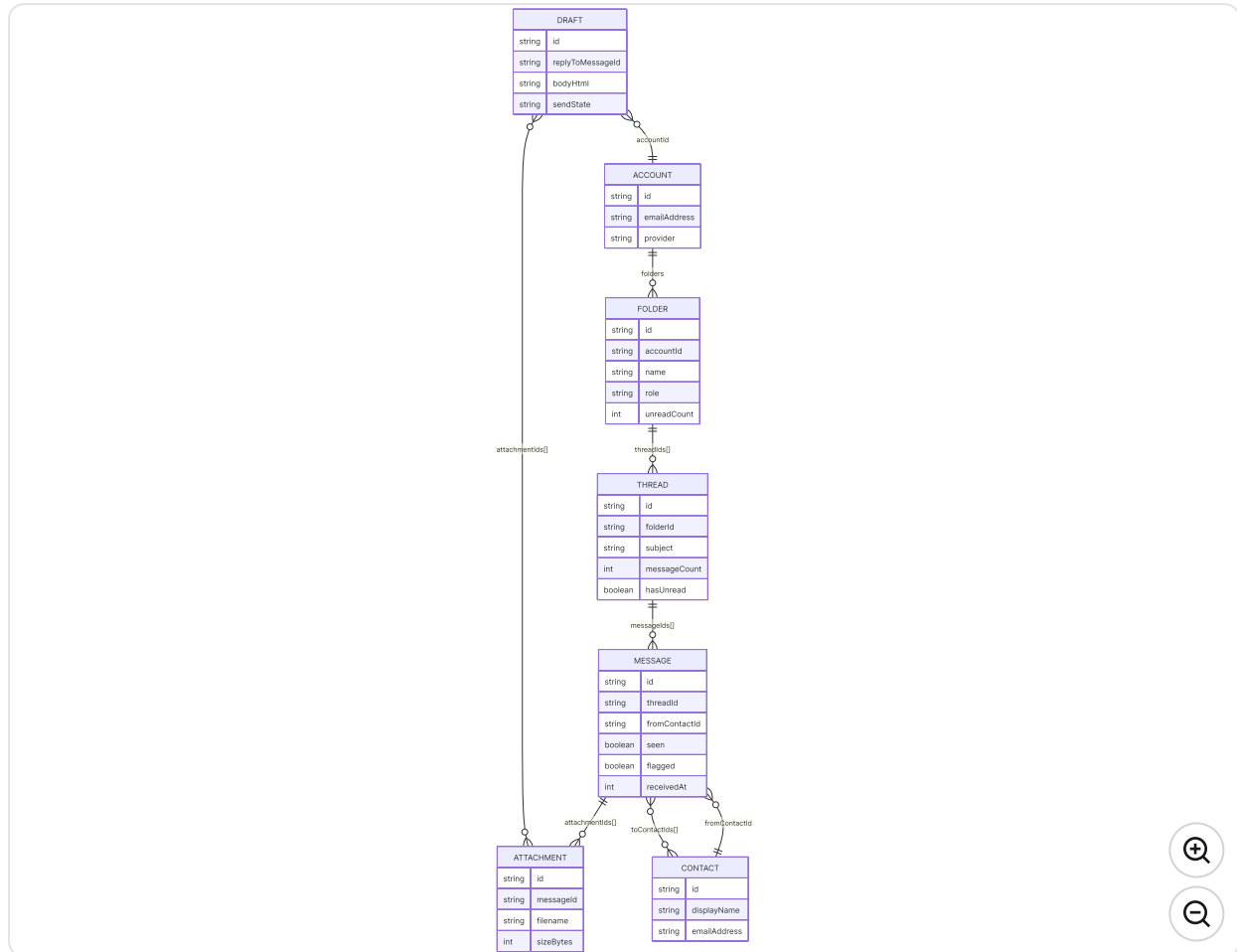


Reveal answer

Check answer ↩

Data model

Work-in-progress



Normalized client store for folders, threads, and messages

Interface definition (API)

Work-in-progress

Optimizations and deep dive

With the core mail surfaces in place, this section focuses on the product details that make an email client usable for daily work:

- **Composing:** This section covers rich text drafting, address autocompletion, attachments, drafts, undo send, and other write-side email flows.
- **Keyboard shortcuts:** This section discusses shortcut behavior for power users and how clients such as Gmail, Apple Mail, and Outlook influence expectations.
- **Undo/redo:** This section explains how centralized actions and history make reversible editing operations easier to support.
- **Performance:** This section covers where performance still matters in mail clients, including long message lists, lazy-loaded tools, and expensive rendering work.

❗ **Treat every incoming email body as untrusted HTML from the public internet**

Messages routinely contain XSS payloads, phishing styling that impersonates trusted senders, and tracking pixels that leak read events and IP addresses. Render email bodies inside a sandboxed iframe with a restrictive CSP, strip or rewrite scripts and event handlers, and proxy remote images through your own server so that senders cannot correlate opens with IPs. Never inject raw message HTML into the main document.

Exercise Beta

An email client renders HTML messages from unknown senders. Which controls reduce script execution, access to the trusted app, and remote-image tracking? Select all that apply.

- 1 Render each message body inside a sandboxed iframe isolated from the main application document. ☐
- 2 Insert the raw message HTML into the application's main document and rely on the application's login boundary for protection. ☐
- 3 Allow direct remote-image requests as long as scripts are disabled in the message body. ☐
- 4 Strip or rewrite scripts, event handlers, and other active message content before display. ☐
- 5 Fetch remote images through an application proxy instead of directly from the message document. ☐

6

Apply a restrictive Content Security Policy to the message document.

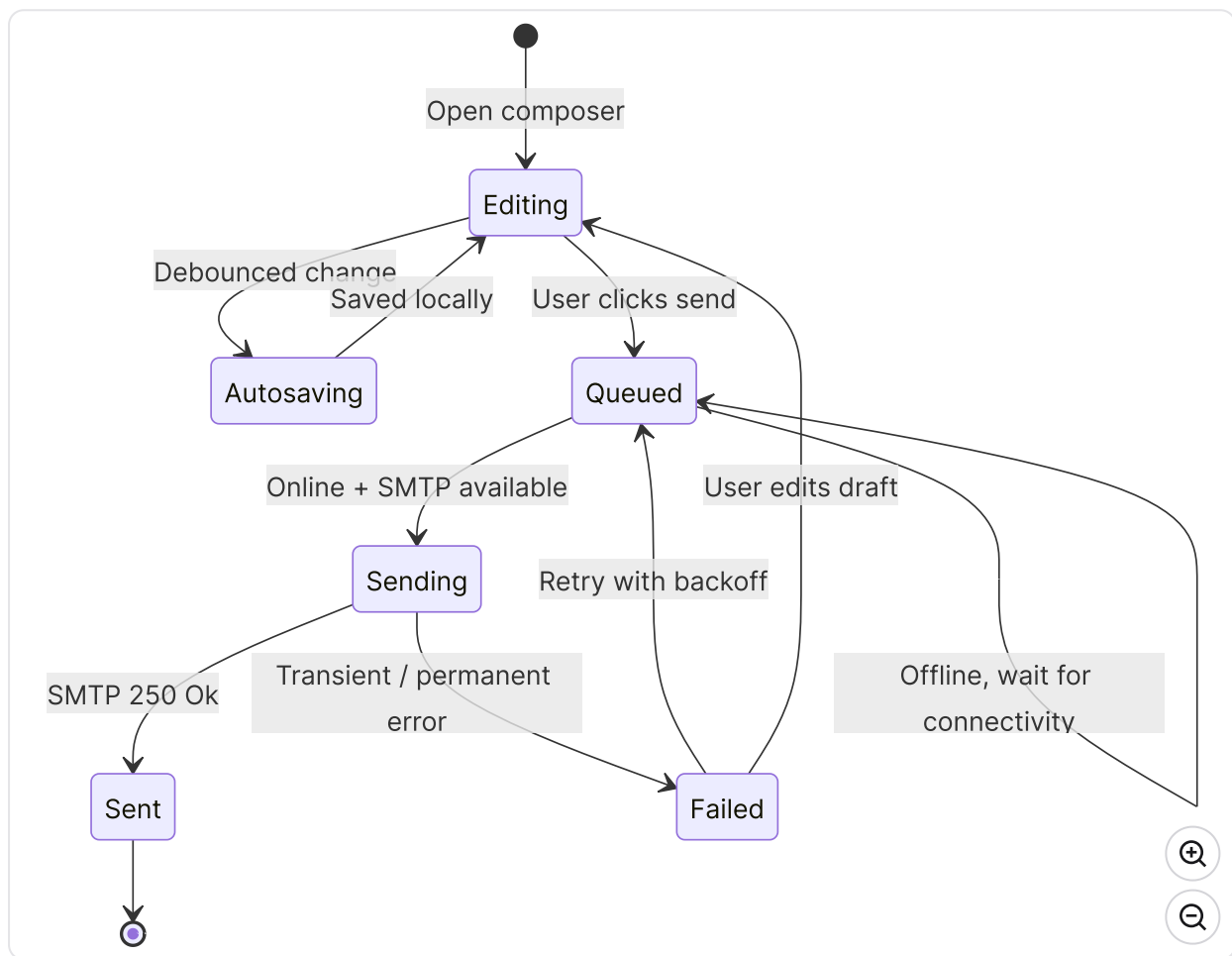


Reveal answer

Check answer ↩

Composing

Work-in-progress

*Draft lifecycle from compose to delivery*

Message format

Work-in-progress

Attachments

- Resume attachment.
- Async attachment flow.

Keyboard shortcuts

- Implement different shortcuts in popular clients like Gmail, Apple Mail, Outlook, etc.

Undo/redo

- Made easy with action history and centralized store.
- Toasts

Performance

- Not that much need for performance.
- Lazy loading is optional and not a priority since the assets are bundled as part of the app. Network request latency is short and only affects JavaScript boot-up and execution time.

Summary

An email client is a synchronization problem dressed as a UI.

Put the sync boundary at the server mailbox. The server mailbox stays the source of truth, which is what gives users a consistent view across phone, desktop, and webmail and lets the provider own backups. The UX layer is a central store plus a persistent task queue on top of a local cache: the store feeds the three-pane view, the cache makes browsing and search work offline, and the queue is the durable channel through which every mutation eventually reaches the server. Read as a candidate, the job is to make that boundary explicit and explain it, not to chase features.

Derive the hard product behaviors from that boundary. Threading by **References** and **In-Reply-To** exists because messages are stored per-message on the server but users reason in conversations on the client. Rendering HTML bodies inside a sandboxed iframe with a restrictive CSP, scripts and event handlers stripped, and remote images proxied is what keeps untrusted server content from escaping into the host document and leaking open events.

Make optimistic mail mutations durable before they leave the client. Archive, flag, move, and send feel instant while the server remains authoritative, which forces the queue to survive restarts, retry with backoff, and reconcile or roll back on rejection. The undo-send window is the same pattern taken seriously: a deliberate client-side delay before SMTP dispatch, not a cancel request to the server.

Let transport and store choices follow the sync thesis. IMAP is the canonical retrieval protocol and SMTP the submission protocol, with POP3 kept only as a legacy fallback. The client logic lives in an isomorphic core with a pluggable DB layer so the same reducers, commands, and sync code can back a native desktop shell or webmail, swapping storage per runtime. The normalized store is shaped around `Account`, `Folder`, `Thread`, `Message`, `Attachment`, `Contact`, and `Draft`, so the message list, reading pane, unread counts, and composer all read from one graph instead of re-deriving state per surface.

If one decision has to lead the answer, it is treating the server mailbox as the sole source of truth and making the local cache plus task queue strictly its optimistic mirror.

References

The references below group email-client sources by the topic they inform, so it is easier to map each source back to the part of the article it relates to: protocols and specs, architecture background, open-source clients, performance case studies, and related system design articles.

Email protocols and specs

This subsection covers the underlying protocols that define how email clients talk to mail servers, which shape the client-side data model and sync behavior.

- [Internet Message Access Protocol \(IMAP\) | Wikipedia](#) for a high-level overview of the protocol most mail clients use to read and manage server-side mailboxes.
- [RFC 3501: IMAP4rev1 specification](#) for the normative spec behind IMAP mailbox, folder, and flag semantics.

Email architecture and background

This subsection provides general-purpose reference material on how end-to-end email works, including the roles of SMTP, POP3, and IMAP.

- [Email architecture, Gmail two-step verification, SMTP POP3 IMAP](#) for an end-to-end tour of how mail servers, clients, and authentication fit together.
- [Email program classifications](#) for the MUA, MTA, and MDA roles that frame where a client sits in the overall system.
- [How does email work? | Namecheap](#) for an accessible walkthrough of the end-to-end mail delivery path.
- [Ninjamail: scalable internet mail service](#) for a research-style look at building a scalable mail service, useful background when reasoning about server-side constraints.

Open-source email clients and case studies

This subsection points at real client implementations and narrative case studies that inform UX, feature set, and engineering tradeoffs for a web-based email client.

- [Nylas Mail \(open-source email client\)](#) for a reference implementation of a cross-platform email client built on web technologies.
- [Mailspring \(open-source email client\)](#) for another actively maintained open-source client showing how features like threading and search are structured.
- [Building my own email app to reach inbox zero | Slate](#) for a narrative on the UX constraints that drive mail app feature choices.
- [Best email clients for Mac | Zapier](#) for a broad feature comparison that helps ground the interview-level feature set.

Performance case studies

This subsection links production performance work on large mail products, relevant to the rendering and data-fetching strategy discussed in the article.

- [Mail.ru optimizes Core Web Vitals | web.dev](#) for how a high-traffic webmail product approached Core Web Vitals and perceived performance.

Related system design articles

The articles below overlap with email on long-lived stateful clients and collaborative or realtime sync, and are useful cross-references when deepening the email design.

- [Chat application \(Messenger\)](#) for another long-lived, list-plus-detail messaging UI with realtime and offline concerns that mirror many mail-client problems.
- [Collaborative editor \(Google Docs\)](#) for the sync and local-state patterns that inform how a mail client reconciles server-side mailbox state with local edits.

Exercises

Exercise 1 of 11 Beta

A team is defining the first version of a desktop email client. Which requirements belong in it? Select all that apply.

- 1

Require several providers to merge into one isolated multi-account inbox.

☐
- 2

Let users browse and search synchronized mail offline, and preserve outgoing messages until connectivity returns.

☐
- 3

Support the major desktop operating systems through a shared product design.

☐
- 4

Require every message to be grouped into a conversation before it can be read.

☐
- 5

Send mail through a configured provider, retrieve new mail, and read messages already stored on the device.

☐



Reveal answer

Check answer ←

Discussions

↩ ↶ | Normal T_T | B I U ↻ | ✎ <> Aa ☰ ☷ </> ☰

Type comment here...

0/1000

Post



Report an issue



Mark complete



Subscribe to our newsletter

Enter your email

Notify me

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community

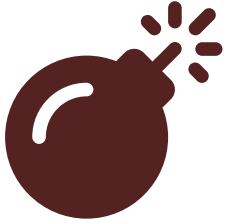


English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

• [Privacy Policy](#)

• [Terms of Service](#)



Syntax error in text
mermaid version 11.14.0